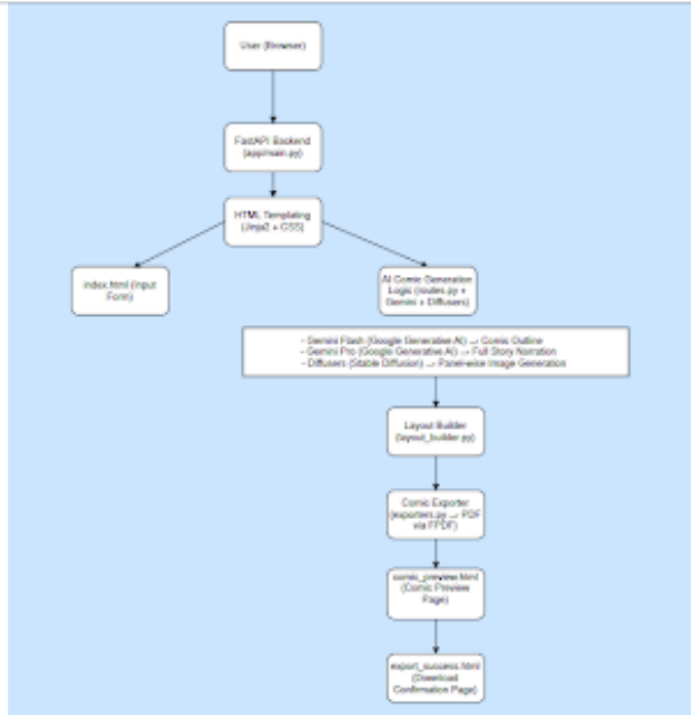


# ComicCraft

## Complete Project Documentation Analysis of Uploaded ZIP + Project Design Document

← ComicCraft - AI Comic Story Creator using Gemini Models.docx



### Pre - requisites:

1. FastAPI Framework Knowledge: [FastAPI Documentation](#)

## AI Comic Story Creator using Gemini Models

FastAPI • Jinja2 • Gemini • Hugging Face Diffusers / Stable Diffusion • FPDF2

## 1. Executive Summary

ComicCraft is a web-based AI comic creator that accepts story details and builds a five-panel comic workflow containing story content, panel illustrations, browser preview and PDF export.

## 2. Source Material

The actual `comic-1.zip` is used for implementation details; the supplied project PDF is used for intended scenarios, architecture and milestones.

### 3. Objectives & Technology Stack

- Create a coherent five-panel comic.
- Generate narration, dialogue and image prompts.
- Preview the comic in the browser.
- Export the comic as PDF.

| Technology    | Role                                                 |
|---------------|------------------------------------------------------|
| Backend       | Python + FastAPI + Uvicorn                           |
| Frontend      | HTML + CSS + Jinja2 templates + JavaScript           |
| Text AI       | Google Gemini through the current `google-genai` SDK |
| Image AI path | Hugging Face Diffusers + Stable Diffusion            |
| PDF           | FPDF2                                                |
| Validation    | Pydantic v2                                          |
| Configuration | python-dotenv + `.env`                               |
| Testing       | pytest + FastAPI TestClient                          |

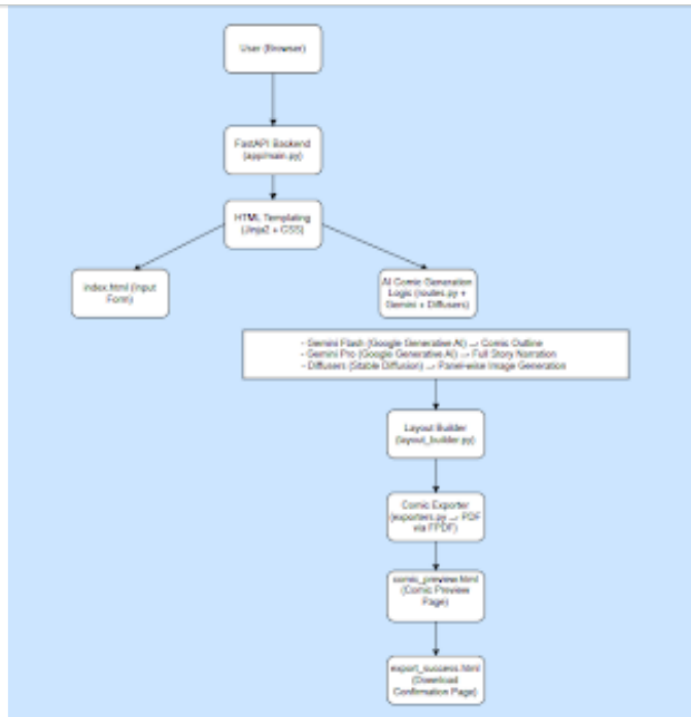
## 4. Actual Project Structure

### ComicCraft • Actual Project Structure from Uploaded ZIP

```
comic/
├─ app/
│   ├─ __init__.py
│   ├─ config.py
│   ├─ exporters.py
│   ├─ gemini_flash.py
│   ├─ gemini_pro.py
│   ├─ image_generator.py
│   ├─ layout_builder.py
│   ├─ main.py
│   ├─ routes.py
│   └─ schemas.py
├─ data/
│   └─ comics/
├─ static/
│   ├─ css/style.css
│   ├─ js/app.js
│   ├─ panels/
│   └─ exports/
├─ templates/
│   ├─ index.html
│   ├─ comic_preview.html
│   ├─ error.html
│   └─ export_success.html
├─ tests/
│   └─ test_smoke.py
├─ .env
├─ .env.example
└─ .gitignore
```

## 5. System Architecture & Data Flow

← ComicCraft - AI Comic Story Creator using Gemini Models.docx



**Pre - requisites:**

1. FastAPI Framework Knowledge: [FastAPI Documentation](#)

- User input → FastAPI → Gemini outline → Gemini story → image generation → layout builder → PDF export → Jinja2 preview.
- The current `routes.py` calls the image generator once for each outline panel.

## 6. API Routes

| Route                | Method | Purpose                      |
|----------------------|--------|------------------------------|
| /                    | GET    | Homepage form                |
| /health              | GET    | Health/status endpoint       |
| /generate            | POST   | HTML form → comic preview    |
| /generate-comic/json | POST   | JSON API → layout + PDF path |
| /test-image          | GET    | Direct image-generation test |
| /export-success      | GET    | Export confirmation          |

# 7. Frontend Visual References

## Homepage

← ComicCraft - AI Comic Story Creator using Gemini Models.docx



**Description:**

The homepage of ComicCraft is designed as an intuitive starting point for users to create AI-generated comic stories. It features a clean layout with a scenic background that fosters a creative atmosphere.

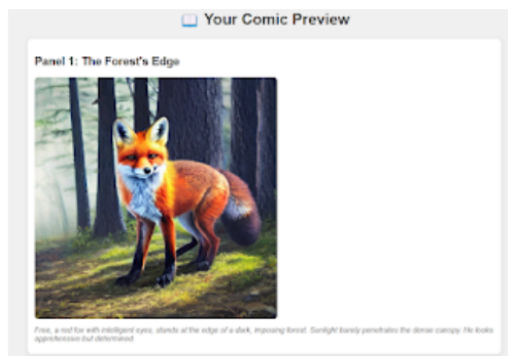
At the center, a structured form collects user inputs necessary for comic generation. Users can:

- Enter a Story Prompt describing their comic idea,
- Specify a Main Character Name,
- Choose a Setting (school, forest, space, city),
- Select a Story Tone (light-hearted, dramatic, poetic, funny),
- Pick an Art Style (anime, pixel art, comic book, realistic).

Once the form is submitted, the system uses AI models to automatically generate a complete comic, including storyline outlines, detailed dialogues, illustrations, and an exportable PDF.

The homepage is designed to be simple, user-friendly, and visually appealing, encouraging creativity and making the comic

## Comic preview



## Comic preview details



### Description:

The Comic Preview Page in ComicCraft presents the AI-generated comic panels sequentially after the user submits their input through the homepage form. This page provides a complete view of the comic's storyline, combining visual and textual elements for each panel.

Each panel displayed on the page includes:

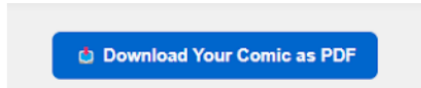
- Panel Title: Clearly labeled with the panel number and a descriptive title (e.g., "Panel 2: Into the Deep Woods").
- Comic Image: An AI-generated illustration created based on the scene's image prompt.
- Scene Description: A short descriptive paragraph, displayed in italics, that sets the atmosphere and context for the panel.
- Caption and Narration:
  - Caption providing a brief, ambient description of the environment or background sounds.
  - Narration detailing the main character's actions, emotions, or dialogue within the scene.
- Image Prompt Reference: The original artistic description used to generate the illustration, displayed for reference.

## Export success

The combination of imagery, scene-setting description, and narrative text helps create an immersive storytelling experience directly from the AI-generated content.

At this stage, the page focuses purely on comic viewing and story review, allowing users to experience the full creative output before proceeding to any final actions like exporting.

**Download Button:**



**Description:**

On the Comic Preview Page, a "Download Your Comic as PDF" button is displayed.

When users click this button, the entire comic — including images, panel titles, scene descriptions, captions, and narration — is compiled into a professionally formatted PDF file and downloaded directly to the user's device.

This ensures users can save, share, and view their comic offline.

**Comic Export Success Page:**



**Description:**

After the download is triggered, users are automatically redirected to the Comic Export Success Page. This page provides a confirmation message indicating that their AI-powered comic has been successfully created and downloaded. The success

# 8. Current Panel Image Evidence

The sampled panel images in the current ZIP show the placeholder image design and the text 'Placeholder image mode'.



## 9. Actual Coding Screenshots

### routes.py

ComicCraft • app/routes.py • Main AI Generation Pipeline

```
1  from __future__ import annotations
2
3  from pathlib import Path
4  from uuid import uuid4
5
6  from fastapi import APIRouter, HTTPException, Request
7  from fastapi.responses import HTMLResponse, JSONResponse
8  from fastapi.templating import Jinja2Templates
9
10 from .config import settings
11 from .gemini_flash import generate_outline
12 from .gemini_pro import generate_story
13 from .image_generator import generate_image
14 from .layout_builder import build_comic_layout
15 from .exporters import save_pdf
16 from .schemas import (
17     ComicOutline,
18     ComicPanel,
19     ComicStory,
20     StoryPanel,
21     PromptRequest,
22 )
23
24
25 router = APIRouter()
26
27 templates = Jinja2Templates(
28     directory=str(settings.TEMPLATES_DIR)
29 )
30
31
32 # =====
33 # HOME
34 # =====
35
36 @router.get("/", response_class=HTMLResponse)
37 async def home(request: Request):
38
39     defaults = {
40         "story_prompt": "",
41         "character_name": "",
42         "setting": "",
43         "tone": "Dramatic",
44         "art_style": "Comic book",
45     }
46
47     return templates.TemplateResponse(
48         request=request,
49         name="index.html",
50         context={
51             "app_name": settings.APP_NAME,
52             "defaults": defaults,
53         },
54     )
55
56
57 # =====
58 # HEALTH CHECK
59 # =====
60
61 @router.get("/health")
62 async def health():
63
64     return {
65         "status": "ok",
66         "mock_ai": settings.ALLOW MOCK_AI,
67     }
68
69
70 # =====
71 # CONVERT DICT -> COMIC OUTLINE
72 # =====
73
74 def _to_comic_outline(data: dict) -> ComicOutline:
75
76     panels = []
77
78     for index, panel in enumerate(
79         data.get("panels", []),
80         start=1,
81     ):
82
83         panels.append(
84             ComicPanel(
85                 panel_number=panel.get(
86                     "panel_number",
87                     panel.get("panel", index),
88                 ),
89                 title=panel.get(
90                     "title",
91                     f"Panel {index}",
92                 ),
93                 scene_description=panel.get(
94                     "scene_description",
95                     panel.get(
96                         "description",
97                         "Comic scene",
98                     ),
99                 ),
100                 image_prompt=panel.get(
101                     "image_prompt",
102                     panel.get(
103                         "description",
104                         "Comic panel",
105                     ),
106                 ),
107             )
108         )
109
110     return ComicOutline(
111         title=data.get("title", "Comic Story"),
112         character_name=data.get("character_name", "Comic Character"),
113         setting=data.get("setting", "Comic World"),
114         tone=data.get("tone", "Dramatic"),
115         art_style=data.get("art_style", "Comic book"),
116         panels=panels,
117     )
```

Coordinates outline → story → images → layout → PDF.

## image\_generator.py

ComicCraft • app/image\_generator.py • Hugging Face / Diffusers

```
1  from __future__ import annotations
2
3  import threading
4  import uuid
5  from pathlib import Path
6
7  from PIL import Image, ImageDraw, ImageFont
8
9  from .config import settings
10
11
12  # =====
13  # GLOBAL DIFFUSION PIPELINE
14  # =====
15
16  _pipeline = None
17
18  _pipeline_lock = threading.Lock()
19
20
21  # =====
22  # FONT HELPER
23  # =====
24
25  def _safe_font(size: int = 26):
26
27      candidates = [
28          "/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf",
29          "C:/Windows/Fonts/arial.ttf",
30      ]
31
32      for candidate in candidates:
33
34          path = Path(candidate)
35
36          if path.exists():
37
38              try:
39                  return ImageFont.truetype(
40                      str(path),
41                      size
42                  )
43              except OSError:
44                  pass
45
46      return ImageFont.load_default()
47
48
49  # =====
50  # PLACEHOLDER IMAGE
51  # =====
52
53  def _create_placeholder(
54      prompt: str,
55      panel_number: int
56  ) -> Path:
57
58      filename = (
59          f"panel_{panel_number}_"
60          f"{uuid.uuid4().hex[:10]}.png"
61      )
62
63      path = settings.PANELS_DIR / filename
64
65      image = Image.new(
66          "RGB",
67          (
68              settings.IMAGE_WIDTH,
69              settings.IMAGE_HEIGHT
70          ),
71          "#e9eef5"
72      )
73
74      draw = ImageDraw.Draw(image)
75
76      title_font = _safe_font(28)
77      body_font = _safe_font(17)
78
79      draw.rounded_rectangle(
80          (
81              18,
82              18,
83              settings.IMAGE_WIDTH - 18,
84              settings.IMAGE_HEIGHT - 18
85          ),
86          radius=24,
87          outline="#172033",
88          width=4,
89      )
90
91      draw.text(
92          (40, 45),
93          f"ComicCraft - Panel {panel_number}",
94          fill="#172033",
95          font=title_font,
96      )
97
98      # Word wrapping
99      words = prompt.split()
100
101      lines = []
102      current = ""
103
104      for word in words:
105
106          test = f"{current} {word}".strip()
107
108          if len(test) > 42:
109
110
```

Contains placeholder mode and the Diffusers/Stable Diffusion path.

## ComicCraft • app/main.py • FastAPI Application

```

1  from __future__ import annotations
2
3  from fastapi import FastAPI
4  from fastapi.staticfiles import StaticFiles
5
6  from .config import settings
7  from .routes import router
8
9
10 # =====
11 # FASTAPI APPLICATION
12 # =====
13
14 app = FastAPI(
15     title=settings.APP_NAME,
16     description=(
17         "Generate personalized five-panel "
18         "AI comics using Gemini and "
19         "Stable Diffusion."
20     ),
21     version="1.0.0",
22 )
23
24
25 # =====
26 # STATIC FILES
27 # =====
28
29 app.mount(
30     "/static",
31     StaticFiles(
32         directory=str(settings.STATIC_DIR)
33     ),
34     name="static",
35 )
36
37
38 # =====
39 # PDF EXPORT FILES
40 # =====
41
42 app.mount(
43     "/export",
44     StaticFiles(
45         directory=str(settings.EXPORTS_DIR)
46     ),
47     name="export",
48 )
49
50
51 # =====
52 # ROUTES
53 # =====
54
55 app.include_router(
56     router
57 )
58
59
60 # =====
61 # FAVICON
62 # =====
63
64 @app.get(
65     "/favicon.ico",
66     include_in_schema=False,
67 )
68 async def favicon():
69     return {}

```

Creates the FastAPI app and mounts static/export directories.

## ComicCraft • app/schemas.py • Pydantic Validation Models

```

1  from __future__ import annotations
2
3  from typing import List
4
5  from pydantic import BaseModel, Field, field_validator
6
7
8  # =====
9  # USER REQUEST
10 # =====
11
12 class PromptRequest(BaseModel):
13     story_prompt: str = Field(
14         ...,
15         min_length=3,
16         max_length=2000
17     )
18
19     character_name: str = Field(
20         ...,
21         min_length=1,
22         max_length=80
23     )
24
25     setting: str = Field(
26         ...,
27         min_length=1,
28         max_length=120
29     )
30
31     tone: str = Field(
32         ...,
33         min_length=1,
34         max_length=60
35     )
36
37     art_style: str = Field(
38         ...,
39         min_length=1,
40         max_length=80
41     )
42
43     @field_validator(
44         "story_prompt",
45         "character_name",
46         "setting",
47         "tone",
48         "art_style"
49     )
50     @classmethod
51     def strip_values(cls, value: str) -> str:
52
53         value = value.strip()
54
55         if not value:
56             raise ValueError(
57                 "This field cannot be empty."
58             )
59
60         return value
61
62
63 # =====
64 # OUTLINE
65 # =====
66
67 class ComicPanel(BaseModel):
68     panel_number: int = Field(
69         ...,
70         ge=1
71     )
72
73     title: str
74
75     scene_description: str
76
77     image_prompt: str
78
79
80 class ComicOutline(BaseModel):
81     panels: List[ComicPanel] = Field(
82         ...,
83         min_length=5,
84         max_length=5
85     )
86
87
88 # =====
89 # STORY
90 # =====
91
92 class StoryPanel(BaseModel):
93     panel_number: int = Field(
94         ...,
95         ge=1
96     )
97
98     title: str
99
100     scene_description: str

```

Defines validated user input and fixed five-panel models.

## ComicCraft • app/gemini\_flash.py • Gemini Outline Generation

```

1  import json
2
3  from google import genai
4  from google.genai import types
5
6  from .config import settings
7
8
9  def _mock_outline(
10     story_prompt: str,
11     character_name: str,
12     setting: str,
13     tone: str,
14     art_style: str,
15 ) -> dict:
16     """Create a local 5-panel outline without calling Gemini."""
17
18     panels = [
19         {
20             "panel": 1,
21             "title": "The Beginning",
22             "description": (
23                 f"{character_name} begins an adventure in {setting}."
24             ),
25             "narration": (
26                 f"{character_name} never expected this adventure to begin today."
27             ),
28             "dialogue": "Something feels different today...",
29             "image_prompt": (
30                 f"{character_name} entering {setting}, "
31                 f"{art_style} comic book style"
32             ),
33         },
34         {
35             "panel": 2,
36             "title": "A Strange Discovery",
37             "description": (
38                 f"{character_name} discovers something unusual in {setting}."
39             ),
40             "narration": (
41                 "A mysterious discovery catches their attention."
42             ),
43             "dialogue": "What is that?",
44             "image_prompt": (
45                 f"{character_name} discovering something mysterious "
46                 f"in {setting}, {art_style} comic book style"
47             ),
48         },
49         {
50             "panel": 3,
51             "title": "The Challenge",
52             "description": (
53                 f"{character_name} faces a challenge during the adventure."
54             ),
55             "narration": (
56                 "The adventure suddenly becomes dangerous."
57             ),
58             "dialogue": "I have to keep going!",
59             "image_prompt": (
60                 f"{character_name} facing an exciting challenge "
61                 f"in {setting}, {art_style} comic book style"
62             ),
63         },
64         {
65             "panel": 4,
66             "title": "The Turning Point",
67             "description": (
68                 f"{character_name} finds a clever way "
69                 "to overcome the challenge."
70             ),
71             "narration": (
72                 "With courage and determination, a solution appears."
73             ),
74             "dialogue": "I know what to do!",
75             "image_prompt": (
76                 f"{character_name} solving a difficult problem "
77                 f"in {setting}, heroic {art_style} comic book style"
78             ),
79         },
80         {
81             "panel": 5,
82             "title": "A New Beginning",
83             "description": (
84                 f"{character_name} completes the adventure "
85                 "and returns safely."
86             ),
87             "narration": (
88                 "The adventure ends, but a new journey is about to begin."
89             ),
90             "dialogue": "This is only the beginning.",
91             "image_prompt": (
92                 f"{character_name} standing proudly in {setting}, "
93                 f"sunset, {art_style} comic book style"
94             ),
95         },
96     ]
97
98     return {
99         "title": story_prompt[:80] or "ComicCraft Adventure",
100         "panels": panels,

```

Generates the structured comic outline.

## ComicCraft • app/gemini\_pro.py • Gemini Story Generation

```

1  import json
2
3  from google import genai
4  from google.genai import types
5
6  from .config import settings
7
8
9  def _mock_story(
10     outline: dict,
11     character_name: str,
12     tone: str,
13     art_style: str,
14 ) -> dict:
15     """
16     Create a local story without calling Gemini.
17
18     Used when ALLOW MOCK AI=true.
19     """
20
21     panels = []
22
23     for index, panel in enumerate(
24         outline.get("panels", []),
25         start=1,
26     ):
27         panels.append(
28             {
29                 "panel": panel.get(
30                     "panel",
31                     index,
32                 ),
33                 "title": panel.get(
34                     "title",
35                     f"Panel {index}",
36                 ),
37                 "narration": panel.get(
38                     "narration",
39                     "The adventure continues.",
40                 ),
41                 "dialogue": panel.get(
42                     "dialogue",
43                     f"{character_name}: Let's keep going!",
44                 ),
45                 "image_prompt": panel.get(
46                     "image_prompt",
47                     panel.get(
48                         "description",
49                         f"{character_name}, "
50                         f"{art_style} comic book style",
51                     ),
52                 ),
53             }
54         )
55
56     return {
57         "title": outline.get(
58             "title",
59             "ComicCraft Adventure",
60         ),
61         "panels": panels,
62     }
63
64
65 def _get_client():
66     if not settings.GEMINI_API_KEY:
67         raise RuntimeError(
68             "GEMINI_API_KEY is not configured."
69         )
70
71     return genai.Client(
72         api_key=settings.GEMINI_API_KEY
73     )
74
75
76 def generate_story(
77     outline: dict,
78     character_name: str,
79     tone: str,
80     art_style: str,
81 ) -> dict:
82     """
83     Generate the final comic story.
84
85     Uses mock data when ALLOW MOCK AI=true.
86     """
87
88     # -----
89     # MOCK MODE
90     # -----
91
92     if settings.ALLOW MOCK AI:
93         return _mock_story(
94             outline,
95             character_name,
96             tone,
97             art_style,
98         )
99
100    # -----
101    # REAL GEMINI MODE
102    # -----
103
104    client = _get_client()
105
106    prompt = f"""
107    You are a professional comic book writer.
108
109    Create the final story for this 5-panel comic.
110
111    Main character:
112    {character_name}
113
114    Tone:
115    {tone}

```

Generates narration, dialogue and visual prompts.

## 10. Testing & Local Run

- Create/activate a virtual environment.
- Install `requirements.txt`.
- Run `uvicorn app.main:app --reload`.
- Open `http://127.0.0.1:8000` and `/docs`.
- Run the smoke tests with `pytest`.

## 11. Implementation Review: Design vs Current ZIP

**Image dependencies:** The design document lists Diffusers/Transformers/PyTorch, while the current ZIP `requirements.txt` does not list those packages.

**Image provider default:** Current `config.py` defaults `IMAGE_PROVIDER` to `placeholder`.

**Image fallback:** Current `config.py` defaults `ALLOW_IMAGE_FALLBACK` to `true`, allowing placeholder output after image-generation failure.

**Gemini configuration:** Current ZIP uses `google-genai` and environment-driven model IDs; the defaults differ from the Gemini 1.5 names in the original design document.

**Panel count:** Current Pydantic models enforce exactly five outline and five story panels.

## 12. Demonstration Checklist

- Show VS Code structure.
- Show `routes.py` and `image_generator.py`.
- Run Uvicorn.
- Open homepage and generate a comic.
- Verify image generation with `/test-image` when using Hugging Face/Stable Diffusion.
- Open the PDF export.

## 13. Conclusion

ComicCraft combines FastAPI, Jinja2, Gemini text generation, a Stable Diffusion/Diffusers image-generation path, layout assembly and FPDF export. The ZIP provides implementation evidence while the project PDF provides the intended design.